大模型协作记录（AI Collaboration Log）

> 竞赛要求：提交**体现大模型协作过程的交互记录，含提示词、模型回答与自我纠错轨迹**（`COMP_GUIDE_2026.txt:807-808`）；报告正文中需有专门章节 「大模型协作记录：提示词、模型回答、自我纠错轨迹、智能体工作流设计（如有）」（`COMP_GUIDE_2026.txt:857`）。
>
> 本文件的目的是把**真实发生过的事情**按可核查的格式归档。三条自我约束：
>
> 1. **每一条都带仓库内锚点**（`docs/DEV_STATUS.md:行号`、源文件行号或产物文件名）。 没有锚点的事件不写进本记录 —— 因为无法验证的"协作故事"没有价值。
> 2. **用户提示词按原话引用**，并标注出处（本仓库里用户原话最权威的三个落点见 §1.2）。
> 3. **记录失败。**本记录里至少 8 条是"我先做错了、然后被实测推翻"。 只写"AI 一次就做对了"的记录，在评审眼里等于没写。

---

1. 协作方式总览

1.1 人与模型的角色分工

角色	承担	具体表现
人（队长）	需求、拍板、物理世界、最终验收	提需求原话；在多个方案里选一个（`m08918` 五项全选"推荐"）；接线与上电；示波器/信号源操作；决定"先做哪一项"
模型（AI 智能体）	设计、实现、取证、自检、归档	读写仓库、跑仿真与实现、跑真机脚本、写文档、自己找自己的 bug
确定性工具	判定（不依赖模型）	`sim/logs/SUMMARY.md`、`impl_timing.rpt`、`*_check.py` 的退出码

最重要的一条约定：凡是"结论"都要落到**不依赖模型立场**的判定上 —— 测试台的 `TEST PASSED/FAILED`、实现的 `All user specified timing constraints are met.`、 `tools/pinplan_check.py` 的 `RESULT: PASS`、真机脚本的 `PASS n/m`。 模型的自述只用于解释，不用于结论。

1.2 用户提示词的三个权威落点（可直接核对）

落点	内容
`docs/10_host_requirements.md:188` 起「附录 A：需求原话（用户 m00001）」	上位机软件的第一版完整需求原话
`_synth_work/prompt_host.md`（8 行）	界面优化六条需求的**全文原话**，与 `docs/12_host_ui_requirements.md` 的 R1–R6 逐条对应；第 7–8 行是后续追加的放大态需求（对应批次 4a/4b）
`_synth_work/prompt_disp.md`（3 行）	7 寸屏本机界面的需求原话 —— ⚠️ **这一条线尚未实现**，本记录只在 §4 当作"下一阶段"提及，**不计入已完成工作**

⇒ 这三份文件本身就是"提示词"的物证：它们是用户输入被原样落盘的产物，不是我事后重写的。

1.3 一个典型回合的流程（本记录的固定行文格式）

① 用户原话（带消息锚点）
② 我的错误假设 / 第一版方案
③ 暴露它的数字或现象（来自实测或仿真，不是来自我的推理）
④ 修法 + 验证（改了什么、用什么证明改对了）
⑤ 可复用结论（已沉淀到 skills/ 的哪一项）

---

2. 自我纠错轨迹（11 条真实记录）

记录 1：测试台"空转"——激励根本没接到被测逻辑上

- ① **触发**：多轮仿真"看起来正常"但覆盖率异常。用户提示词原文见 `docs/DEV_STATUS.md:465-493`（§1.11）。
- ② **错误假设**：我默认 `tb_*` 里的激励已经接到了 DUT 端口上，于是把注意力放在 DUT 逻辑上。
- ③ **暴露它的现象**：写了一个**只找"被计算但从未被读"的线**的静态检查（`tools/scan_dangling.py`），它列出测试台内部的中间信号 —— 激励驱动的信号 **从未出现在任何 DUT 端口连接里**。也就是说那个测试台一直在自己跟自己玩。
- ④ **修法 + 验证**：修测试台连接，重跑；同一批测试台从"看似全绿"变成真正跑通。 此后 `tools/scan_dangling.py` / `tools/scan_unused.py` 成为固定检查项， 与回归并列跑（见 `skills/vivado-report-autocollect.md` §3.4）。
- ⑤ **可复用结论**：**"测试通过"必须先证明"测试真的在测东西"**。 静态检查（死信号）能抓住断言永远抓不到的东西。 ⇒ 沉淀进 `skills/testbench-blindspot.md`（"先怀疑工具与测试台，再怀疑逻辑"）。

记录 2：工具真 bug 污染了实验数据 —— 该先怀疑工具

- ① **触发**：真机读回的寄存器值与写入值不符，一度被当作"RTL 有 bug"。
- ② **错误假设**：先怀疑 RTL。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:3549-3552`（§1.68 D）—— 调试脚本 `0x051A` 的位域被**截断成 4 位**；并且"哨兵写 `0x00FF` "这个动作 在 RTL 里被解释成 **粗调 ×8**。两条都是**上位机脚本自己的 bug**。
- ④ **修法 + 验证**：修脚本位宽与哨兵值，重读，RTL 行为与手册一致。
- ⑤ **可复用结论**：**"工具真 bug 污染实验数据"是高概率事件**。 修法顺序：先证伪工具 → 再证伪测试台 → 最后怀疑被设计对象。 ⇒ 沉淀进 `skills/testbench-blindspot.md`。

记录 3：集成路径 Z 脉宽被窗截断 —— 单元测试台全部"假通过"

- ① **触发**：Z 输出的脉宽与理论值不符。
- ② **错误假设**：单元测试台（`tb_abz_jitter` 等）已覆盖 Z 宽度，于是从别处找原因。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:2200-2223`（§1.44）—— **单元测试台量到 2 个时钟拍，集成路径实测 2042 拍，理论值 2048 拍**。 根因是集成路径里 Z 被 `z_window` 窗**截断**（`z_window = (cycle_cnt[5:0] == 0)`， `rtl/enc_top.v:643-644`），而单元测试台**没有这个窗**，于是单元台全部"假通过"。
- ④ **修法 + 验证**：把结论标注为"**只在窗常开下成立 / 集成路径 v7 实为 40 ns，v8 起才成立**"， 并同步改掉 5 处受影响文档（`docs/DEV_STATUS.md` §1.26 的 `z_width=4 ⇒ 3.9 µs` 表、 §1.33 ② 的 `tb_abz_jitter` "Z 宽度 3.84 µs"、 `docs/06_board_test_guide.md:293`、 `docs/05_vivado_sim_impl_report.md:153` 与 13

项表第 13 条）。 ★ 并诚实记下：`docs/DEV_STATUS.md` §1.42 那次代码审计**没有发现这条截断**， 因为它只比对代码与注释，没有跨"单元台 vs 集成台"比对。

- ⑤ **可复用结论**：**单元测试全绿 ≠ 集成正确**； 凡是被上层信号门控的时序参数，必须在**集成台**上再量一次。 ⇒ 沉淀进 `skills/testbench-blindspot.md`。

记录 4：ABZ 边沿抖动量化 + "过速静默且无告警"的监视盲区

- ① **触发**：用户提示词原文要求量化 AB 抖动并解释过速行为 （`docs/DEV_STATUS.md:1614-1681`，§1.33；触发消息 `m02801④` / `m00598`）。
- ② **错误假设**：我起初只打算"加大迟滞 `hyst`"来压抖动（这是规范里的常规建议）。
- ③ **暴露它的数字**：量化后发现 —— 规范里"加大 hyst"的建议**只在"角度干净、抖动纯属量化"时才成立**；本工程设计里角度本身有噪声，加大 hyst **无益**。 同时发现"过速（UVW 频率告警阈值以上）时输出静默且没有任何告警位"这一监视盲区。
- ④ **修法 + 验证**：把量化依据写进文档，并**明确推翻**自己此前引用的常规建议 （`docs/DEV_STATUS.md` §1.40 ③ 修正 §1.39 的结论）； 新增 `uvw_freq_warn` 类告警位 （帧 off10 的 `io_flags` bit3）进遥测帧，使盲区可观测。
- ⑤ **可复用结论**：**引用手册建议前，先验证该建议的前提在本设计里是否成立。**

记录 5：位域解码口径写错 ⇒ 20 轮统计全部失真

- ① **触发**：一段连续 20 轮的真机统计结果。
- ② **错误假设**：把这 20 轮当作设计质量的证据。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:1362`、`:1376-1393` —— 原文写着"**我的解码器当时写错了**"：位域解码口径错了，所以那 20 轮的统计**全部失真**， 不能作为结论。
- ④ **修法 + 验证**：修解码器、重跑 20 轮，重出统计。
- ⑤ **可复用结论**：**统计结果的可信度上限 = 解码口径的正确性。** 凡是"批量统计得出一个漂亮结论"，先拿 1 帧**逐位手工对照**一次。

记录 6：时序约束口径 —— 负余量是"诚实建模"的结果，不是设计错误

- ① **触发**：实现报告出现 `Timing constraints are not met.`
- ② **错误假设**：第一反应是"设计没收敛，得降频"（并把"降到 25 MHz"写成了回退项）。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:846-885`（§1.19）—— ADC 源同步输入路径按数据手册诚实建模：`set_input_delay -max 14.000/-min 9.500`， 给定源时钟 16.78 ns、总预算 19.78–23.78 ns，得到 **WNS −3.271 ns / 26 个失败端点 全部是 ADC 输入路径。** ⇒ 这个负值是"把真实最坏情况写进约束"的必然结果， **不是综合器失误。**
- ④ **修法 + 验证**：不掩盖：报告里写明"负值里约 3 ns 是相位口径错误，其余是真实建模余量"， 并给出实证回退顺序（`constr/enc_top.xdc:243-259` 五步）： ① 缩短排线 / 串 22~33 Ω；② 捕获改 `clk_i` 下降沿；③ 降采样率；④ 最后才 `set_false_path` （必须写明"ADC 输入路径未做时序分析"）。 ★ 最终由**上板实测**给出结论：示波器量 `tPD_meas = 3–4 ns`（手册 2…6 ns 内）， 建立余量 ≈ **+12.3 ns** ⇒ **①②③④均确认不需要执行**， 同相捕获 + `-max 14.000/-min 9.500` 已收敛（`sys_clk` **WNS +0.959 / 0 失败端点**）。
- ⑤ **可复用结论**：**"未达标"要分清是"设计没做到"还是"约束写得诚实"**， 并且**用一句可量测的上板判据**（这里是 `tPD_meas ≤ 4 ns`）把争论终结。 ⇒ 沉淀进 `skills/adc-source-sync-constraints.md`。

记录 7：环境类假失败三连 —— 残留进程 / 假成功退出 / 挂死

- ① **触发**：同一脚本、同一 RTL，有时通过有时"失败"。
- ② **错误假设**：先怀疑 RTL 改坏了。
- ③ **暴露它的现象**（三条，全部有记录）： 1. `docs/DEV_STATUS.md:26-29`：首轮回归报 `ERROR: [XSIM 43-3356] Could not open file xsim.dir/tb_enc_top_sim/xsim.type for writing` —— 根因是**上一轮被中断的 xsim 进程仍占用该目录**（PID 29468 / 28132）； 杀进程后单跑立即 PASS（497 s）。 ⇒ 由此给**每个测试台分配独立工作目录**（`_sim_work/<tb>/`）。 2. `docs/DEV_STATUS.md:2278`： `vivado` 在打印 `Bitgen Completed Successfully` 之后 **不再退出**（17 分钟无新输出）⇒ 按 `.bit` 与五份 `impl_*.rpt` 的**时间戳与哈希** 确认写完后手工终止 —— 脚本"几乎成功"其实就是成功。 3. `docs/DEV_STATUS.md:2534` / `:2537` / `:3186`：受限环境下 `opt_design` 展开 debug core **挂死**（`rundef.js(122,5) Access denied`，两个 OOC run 子进程 0% CPU 死等 11 分钟）； 另有 `Get-PSDrive` 报 0 GB 而实际 29 GB 可用的假信息。 ⇒ ILA/生产位流构建改用全权限跑，同一脚本一次通过。
- ④ **修法 + 验证**：把这三条写进工具文档，并把"**用产物时间戳与哈希判断构建是否真的完成**" 固化成流程（见 `skills/vivado-report-autocollect.md` §3.2）。
- ⑤ **可复用结论**：**脚本 `exit 1` 不等于构建失败；脚本 `exit 0` 也不等于成功。** 判据是产物。

记录 8：我自己的预测被自己的实测证伪（诚实记录，结论悬置）

- ① **触发**：对 `angle_raw` 毛刺的来源给出过一个几何假设（"Gauss 格点"）。
- ② **错误假设**： `docs/DEV_STATUS.md:2701-2710` 的预测 —— 毛刺与某个格点几何关系一一对应。
- ③ **暴露它的数字**： `docs/DEV_STATUS.md:2747-2753`、 `:2828-2831` —— 用 4 版 `ang_glitch` 触发窗口 + 环路再注入对比 + v6 复算， **"我上一节的预测被证伪（诚实记录，不许糊弄）"**。
- ④ **修法 + 验证**：改为**三轮修正后把结论悬置**，明确列出仍需的激励 （`crcerr` 仍是开放项；板上 `ma` 无主站时钟 ⇒ `0x053A` bit7/bit6 恒 0，是平凡结果； 要真正激励须按 `docs/08_bissc_dumb_host.md` 的方案 A/B 复测，**需用户改接线**）。
- ⑤ **可复用结论**：**结论悬置是一种合法产出。** 写下"我还需要什么证据才能定论"，比编一个自洽的故事有价值。 ⇒ 沉淀进 `skills/adc-source-sync-constraints.md` （"判据要可量测"） 与 `skills/deterministic-measurement.md` （"证据不足时结论悬置"）。

记录 9：子代理分片审计 —— 23 个文件 / 6 个子代理 / 26 处真缺陷

- ① **触发**：用户提示词原文（`m03503`）要求做一次全面自查，并亲自给了分片清单（`docs/DEV_STATUS.md:3175-3199`，§1.66）。
- ② **我的做法**：把 23 个文件切成 6 片，**每片一个独立子代理**， 子代理只负责"找问题并列证据"，**不允许改代码**；我再逐条复核、只修真的。
- ③ **暴露的数字**：6 片合起来报了一批疑点，逐条复核后确认 **26 处真缺陷**， 包括： `encoder` 丢帧率口径错、 `link` 命令 0.502 s/条且丢命令、 `regs` 读失败被当作 0 覆盖、 `disconnect_port` 丢尾巴、 当时叫 `signal.py` 的 `evaluate` 崩溃（该文件后来改名 `alarm.py`）。
- ④ **修法 + 验证**：逐条修复并重跑；报告 `docs/11_host_selfcheck_s1_s5.md`， 逐条证据 `docs/DEV_STATUS.md` §1.66；真机 S1–S5 全套复跑保持全绿。
- ⑤ **可复用结论**：**分片 + 独立评审 + 逐条复核**是"AI 自查"唯一有效的形态：
- 分片让每片的上下文足够小，评审才认真；
- "只找问题、不改代码"避免审计者顺手改坏；
- 复核环节必须有，因为子代理会报**误报**（本次报了不止 26 条）。 ⇒ 沉淀进 `skills/deterministic-measurement.md` 与 `skills/testbench-blindspot.md` （"谁有权下结论"与"怎么证明检查真的在检查"）。

记录 10：打包版启动即崩 —— `host/signal.py` 遮蔽标准库 `signal`

- ① **触发**：用户原话（`m07281` / `m07427` / `m07468`）：源码跑得好好的， **打包成 exe 之后启动就崩**。用户提示词原文见 `docs/DEV_STATUS.md:4331-4334`（§1.77）。
- ② **错误假设**：先怀疑 PyInstaller 打包参数、缺依赖。
- ③ **暴露它的原因**：`host/signal.py` 这个文件**遮蔽了 Python 标准库的 `signal` 模块** —— 源码模式下启动路径不同所以没暴露，打包后 `signal` 被优先解析到本仓库文件， 标准库功能失效 ⇒ 启动即崩。
- ④ **修法 + 验证**：文件改名 `host/signal.py` → `host/alarm.py`， 并同步所有 import；重新打包 + 目标机双击验证通过（S6 验收）。 ★ 这个改名在本次会话里还能在 `git status` 里看到： `RM host/signal.py -> host/alarm.py`。
- ⑤ **可复用结论**：**不要用标准库模块名当自己的文件名**（`signal` / `types` / `queue` / `select` 都是高危名）。这个坑只在"打包 + 干净环境"下暴露 ⇒ 必须真的在目标机验证。 ⇒ 沉淀进 `skills/repo-reproducibility.md` §3.2（单一入口与环境一致性）。

记录 11：把"厂商手册移出仓库"做成了假动作 —— `.gitignore` 对已跟踪文件无效

- ① **触发**：用户原话（`m00444`）：**"先做1，再做2"** —— ① 加 `LICENSE`(MIT) **+ 把厂商手册移出仓库**；② 仓库内文件名去中文。
- ② **错误假设**：我把厂商 PDF / 引脚表 `git mv` 进 `_refs/vendor/`、再往 `.gitignore` 加一行 `_refs/`，就认为"已经移出仓库、不随仓库分发"，并把这句结论写进了 `NOTICE`、`README.md` 与 `docs/DEV_STATUS.md`（第 8 条）。 ★ 我甚至**在 README 自查清单里写了正确的判据** `git ls-files "*.pdf"`，**但从没跑过它。**
- ③ **暴露它的原因**：用户后来让"一起 `git add -A`"（`m01265`），我在清点暂存内容时才发现 `git ls-files _refs` **列出了 6 个厂商文件。**根因：**`.gitignore` 只影响"未被跟踪"的文件**， 这些文件自最初那个基线提交（`9e9aebc`，2026-10-09）起就一直被 git 跟踪， 所以"文件不在仓库里"这句话当时是**假的。**
- ④ **修法 + 验证**：`git rm -r --cached _refs/vendor`（**保留本地文件**，不删工作区）⇒ `git ls-files _refs` 现在是**空输出**、`git check-ignore -v` 命中 `.gitignore:31`、 两个校核脚本（`tools/pinplan_check.py`、`tools/pinout_verify.py`）仍 **RESULT: PASS**（本地资料还在，功能不受影响）。 ⚠️ **只剩一半没解决**：文件**仍在历史提交 `9e9aebc` 里**（`git ls-tree -r 9e9aebc` 能列出）， 所以**克隆仓库依然会下载到厂商手册。**本仓库**没有任何 remote**、共 103 个提交， 重写历史安全但会改掉其后全部哈希 ⇒ 我把"**推到 Gitee/GitHub 之前必须重写历史**" 写成 `NOTICE` §1.1 的显式前置条件与可照抄命令，**没有擅自执行。 ★ 2026-10-15 当晚用户拍板"重写历史"，于是已执行**：先备份 `git bundle create _refs/backup_before_filter.bundle --all`（72 MB， `.gitignore` 排除， 只留本机不随仓库分发）、再 `git filter-branch --index-filter "git rm -r --cached --ignore-unmatch _refs TW28_datasheet_D5en.pdf 'HSTAD9238模块说明书.pdf' AD9238V11标准版原理图.pdf Davinci_Lite_1V0.pdf '双AD9238标准版引脚映射表.xls' '达芬奇Lite开发板IO引脚分配表.xlsx'" --prune-empty --tag-name-filter cat -- --all`（exit 0， **103 个提交全部重写**）。复核：`git rev-list --objects main | grep -iE 'TW28_datasheet|HSTAD9238|AD9238V11|Davinci_Lite|_refs/vendor|引脚映射表|IO引脚分配表'` = **空输出**、 ★ 判据里的 `TW28` 必须写成 `TW28_datasheet`： 仓库里本来就有我们自己写的 `docs/TW28_digest.md`（厂商手册**数值摘录**，按 `NOTICE` §1 不适用 MIT 但确实留在仓库里）， 用宽松的 `TW28` 会把它一起打出来，判据就永远不可能是空 —— **这条一开始我就是写错的， 跑判据时才发现**（判据本身也要被检查）。 新基线 `8f194eb`（旧 `9e9aebc`）、新 tip `6a995e9`（旧 `de59811`）、 `git ls-files` 非 ASCII = 0、`.git` **135 MB → 70 MB。** 遗留一个纯本机的现象：链接工作树 `_v11_verify`（detached HEAD 停在重写前的旧提交） 让那些 blob 仍留在**本机**对象库里 —— 但这**不影响克隆结果**（克隆只取 `main` 可达对象）， 已写进 `NOTICE` §1.1 并给出 `git worktree remove --force _v11_verify` + `git gc` 的收尾办法。 重写还有一个**操**

作教训：我一开始额外建了 `backup-before-filter` 分支再执行 `--all`， 结果 filter-branch **把这个备份分支本身也一起重写了**（`--all` 连它的 ref 都算进去了）， 备份形同虚设；**真正的备份必须是仓库外的 bundle 或外部副本**， 不能是同仓库里的一个分支/ref。

- ⑤ **可复用结论**：**"移出仓库" ≠ "加 `.gitignore`"**，必须 `git rm --cached`（保留本地文件）才真的从索引移除； `.gitignore` 是给"还没被跟踪的东西"用的。 更普适的一条：**写了判据就要立刻跑判据** —— 这条错误的生命周期完全取决于我什么时候第一次 执行 `git ls-files`。⇒ 沉淀进 `skills/repo-reproducibility.md` §3.4（合规判据清单）。

---

3. 智能体工作流设计

3.1 三层结构

```
第 1 层  主智能体（人机对话）
         持有需求、做取舍、写文档、对用户负责
            │
第 2 层  子代理（subagent）— 上下文隔离的专项工作
         例：审计一个文件、挖一段历史、核对一批锚点
         产出"结论 + 证据锚点"，不产出"我觉得"
            │
第 3 层  确定性工具（脚本）— 唯一可作结论的判定来源
         sim/logs/SUMMARY.md、impl_timing.rpt、*_check.py 的退出码、
         真机 selftest 的 PASS n/m、产物时间戳与 SHA256
```

为什么要有第 2 层：一次"审计 23 个文件"的任务如果全在主线上下文里做， 到第 10 个文件时我已经在敷衍自己的检查清单了。分片给独立子代理后， 每片的上下文都足够小，评审质量显著上升（记录 9 的 26 处真缺陷就是这么找出来的）。

为什么必须有第 3 层：模型（包括我）会自我确认。把"结论权"交给退出码与报告文件， 是这套工作流里最重要的一个设计决定。

3.2 子代理的四个使用约定

1. **任务自包含**：子代理看不到主对话，所以提示词里必须给全路径、判据与产出格式。
2. **只做一件事**：一个子代理 = 一个文件或一个主题；不要让它"顺便"做别的。
3. **必须带锚点**：要求它给出 `文件:行号`，并要求它**用 read 逐条复核行号命中**， 不允许只凭 grep 结果就下结论。
4. **审计类子代理禁止改代码**：只出报告，改由主智能体逐条复核后执行（记录 9）。

3.3 上下文管理（长会话的工程化做法）

本项目是长周期开发（30+ 天、多次上板）， 单次会话上下文必然不够。做法：

- **结论外置**：所有结论写进仓库文件（`docs/DEV_STATUS.md` 是主线流水账，`rtl/SPEC_interface_contract.md` 是接口基准），而不是留在对话里。 ⇒ 会话可以断，工程不断。
- **消费过的长输出压缩成摘要**：长的工具输出（仿真日志、PDF 提取文本）在被消费后 压成"结论 + 锚点 + 文件路径"，需要时再回原始文件查。

- **每个动作留时间戳与哈希**：位流、报告、日志都带时间戳，`git log` 提供另一条时间轴。

3.4 本工作流的两个真实代价（诚实记录）

1. **子代理会误报**：记录 9 里子代理报的疑点**多于**26 条，多出来的是误报。 ⇒ 复核环节不可省，不能用"子代理说有问题"直接改代码。
2. **AI 会写出看起来对、实际上是错的文档**：例如把"总输入延迟 19.5…24 ns"当成 AD9238 手册的 `tPD`（真实 `tPD` 是 2…6 ns）写进 5 处文档。 发现方式不是再审一遍文档，而是**回到一手资料**（手册）与**一手实测**（示波器）。 ⇒ 所以本项目有一条硬规定：**报告里的每个数字旁必须标 `[实测]` / `[仿真]` / `[手册]`。**

---

4. 尚未完成 / 下一阶段的协作线

线	状态	说明
7 寸屏本机界面	**需求已收集，未实现**	原话在 `_synth_work/prompt_disp.md`；用户明确要求"先沟通确认，再做下一步执行"；需求文档 `docs/13_display_ui_requirements.md` 已成文
千兆以太网上位机通道	**可行性分析完成，未实现**	`docs/dual-channel-host-feasibility.md`（v2）；结论：技术上可行（复用官方 RGMII 层，3–5 天），但**不解决 10 kSa/s 这个根问题**，且只落在"性能与资源优化效果（20 分）"那格
`crcerr` 的定论	**结论悬置**	需要按 `docs/08_bissc_dumb_host.md` 方案 B 复测（`ma` 开到 10 MHz 故意越界），**需用户改接线**，且会把状态位粘住 ⇒ 排在最后做
BiSS 探针 B / C 在 v12 上复测	**待做**	A 已在 v12 通过（`RTL_BUILD_ID` = 12，`frame_active` 198/200，`crcerr` 0/200）

---

5. 结论：这套协作方式真正起作用的地方

1. **不是"AI 写得快"，而是"AI 愿意反复怀疑自己"。** 本记录 11 条里有 9 条是自我纠错，而每一条的修正都不是靠"再想一遍"， 而是靠**一次新的实测或一个新增的静态检查**把错误暴露出来。
2. **判定权必须交给确定性工具。**人（含 AI）的自述只解释，不下结论。
3. **失败必须入档。**被证伪的预测、被推翻的手册建议、被自己抓住的解码错误 —— 这些恰恰是"设计过程中的判断依据"（竞赛明确要求写进报告， `COMP_GUIDE_2026.txt:825`），也是最难造假的一部分。
4. **沉淀成技能包才有复用价值。**见 `skills/README.md`： 7 项技能全部来自本记录的失败与观察。